

实验六 控制结构程序设计

一、 实验目的：

1. 学会自定义函数的声明、定义和调用，掌握使用自定义函数编写程序。
2. 理解函数的实参、形参及函数返回值的概念及使用；
3. 理解递归思想、递归算法、掌握递归函数的编写；

二、 实验要求：

1. 用 C 语言实现源程序的编写；
2. 源程序应书写规范，适当添加注释；
3. 实验课前完成实验内容源程序的初稿；实验课对源程序进行编辑、调试、运行；实验课后尽快完成实验报告，并按规定时间上交实验报告。

三、 实验源代码、结果及流程图：

1. 利用函数计算素数个数并求和：输入 2 个正整数 m 和 n ($1 \leq m, n \leq 500$)，统计并输出 m 到 n 之间的素数的个数以及这些素数的和。素数就是只能被 1 和自身整除的正整数，1 不是素数，2 是素数。要求定义并调用函数 prime(m) 判断 m 是否为素数，当 m 为素数时返回 1，否则返回 0。

```
#include <stdio.h>
#include <math.h>

//c 语言类型
_Bool isPrime(int n) {
    if(n == 1) //1 不是素数
        return 0;
    for(int i = 2; i <= sqrt(n); i++)
        if(n % i == 0) //不是素数
            return 0;
    return 1;
}

//1 100
int main(int argc, char *argv[]) {
    int m, n;
    scanf("%d %d", &m, &n);
    //内存的利用
    int sum = 0;
    int cnt = 0;
    for(int i = m; i <= n; i++) {
        if(isPrime(i)) { //如果是素数，那么记录到 sum 中，计数器
```

自增

```
        sum += i;
        cnt ++;
    }
}
printf("[%d,%d]区间内的素数个数是:%d\n 总和是:%d\n", m, n, cnt,
sum);
}
```

测试结果:

1 100

[1,100]区间内的素数个数是:25

总和是:1060

2. 使用函数统计指定数字的个数:读入一个整数,统计并输出该数中"2"的个数。要求定义并调用函数 countdigit(number,digit), 它的功能是统计整数 number 中数字 digit 的个数。例如,countdigit(12292,2)的返回值是3。试编写相应程序。

```
#include <stdio.h>
```

```
//功能模块化~ (函数封装)
```

```
int countDigit(int num, int digit){
    int cnt = 0;
    int tmp; //tmp function : record every digit
    while(num){
        tmp = num % 10;
        num /= 10; //退位
        if(tmp == digit)
            cnt++;
    }
    return cnt;
}
```

```
//12292 2
```

```
int main(int argc, char *argv[]) {
    int n, m;
    scanf("%d %d", &n, &m);

    printf("%d 中%d 的个数是%d\n", n, m, countDigit(n, m));
}
```

}

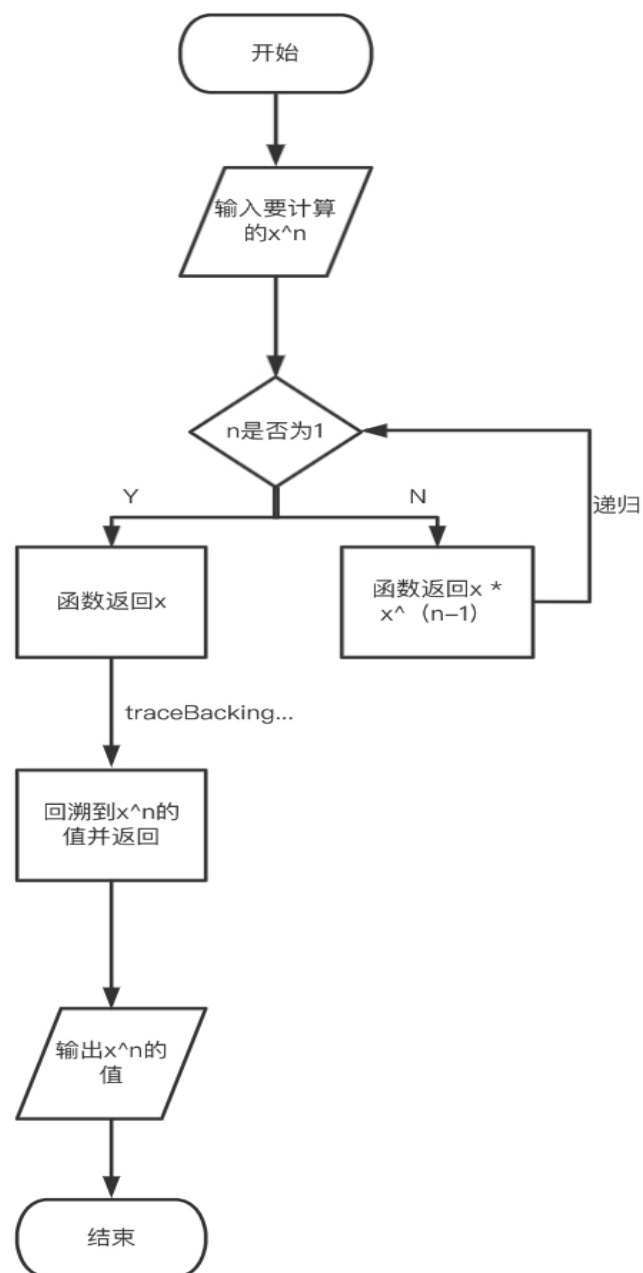
测试结果：

12292 2

12292中2的个数是3

3. 递归实现计算 x^n 。输入实数 x 和正整数 n ，用递归函数计算 x^n 的值。试编写相应程序。用递归函数计算 $x^n (n \geq 1)$ 的值（本题要求在实验报告中画出流程图）。

程序流程图：



```

#include <stdio.h>

//简单递归实现  $x^n$ 
int _power(int x, int n){
    if(n == 1) //递归出口
        return x;
    return x * _power(x, n-1);
}
//递归和回溯的相辅相成~
//2 10 expected : 1024
int main(int argc, char *argv[]) {
    int x, n;
    scanf("%d %d", &x, &n);

    printf("%d 的 %d 次幂的值是 %d", x, n, _power(x, n));
}

```

测试结果:

2 10

2的10次幂的值是1024

4. 递归实现十进制转二进制：输入一个正整数 n ，将其转换为二进制后输出。要求定义并调用函数 `dectobin(n)`，它的功能是输出 n 的二进制数。试编写相应程序。

```

#include <stdio.h>

//递归实现时十进制转化为 二进制
void decToBin(int dec){ //输入一个 10 进制数
    if(dec == 1) { //递归出口
        printf("1");
        return ;
    }
    // while(dec){
    //     tmp = dec % 2;
    //     dec = dec / 2;
    //     printf("%d", tmp);
    // }

    decToBin(dec/2); //向上递归
    //向下回溯(回溯法的体现, 类似栈的逆向输出)
    int tmp = dec % 2; //二进制体现
}

```

```

        printf("%d", tmp);
    }
    //10 expected: 10000000000 (2的10次方, 即1后面应该是10个0)
    int main(int argc, char *argv[]) {
        int dec;
        scanf("%d", &dec);
        printf("%d's Binary form is :", dec);
        decToBin(dec);
    }

```

测试结果:

10

10's Binary form is :1010

5. 编写一个函数, 利用参数传入一个3位数 number, 找出 101~number 之间所有满足下列两个条件的数: 它是完全平方数, 又有两位数字相同, 如 144、676 等, 函数返回找出这样的数据的个数。请同时编写主函数。

```

#include <stdio.h>
#include <math.h>

_Bool judgeSquare(int n){
    if(floor(sqrt(n)) == sqrt(n)) //是完全平方数
        return 1;
    return 0;
}

int digit[3];

_Bool judgeDigit(int n){
    int cnt = 1; //记录数字相同的位数
    int i = 0;
    while(n){
        digit[i++] = n % 10;
        n /= 10;
    }
    for(int i = 0; i < 3; i++){ //循环遍历二维数组
        for(int j = i + 1; j < 3; j++){
            if(digit[i] == digit[j]){
                cnt ++;
            }
        }
    }
}

```

```

    }
    }
    if(cnt == 2) //满足两位数字相同
        return 1;
    }
    return 0;
}
//101 999
int main(int argc, char *argv[]) {
    int x, m;
    scanf("%d %d", &x, &m);

    int cnt = 0;
    printf("[%d, %d]中既是完全平方数，又有两位数字相同的数字如下:\n", x, m);
    for(int i = x; i <= m; i++){
        if(judgeSquare(i) && judgeDigit(i)){
            printf("%d ", i);
            cnt ++;
        }
    }
    printf("\n共有%d个\n", cnt);
}

```

测试结果：

101 999

[101, 999]中既是完全平方数，又有两位数字相同的数字如下：

121 144 225 400 441 484 676 900

共有8个

6. 递归实现顺序输出整数：输入一个正整数 n，编写递归函数实现对其进行按位顺序输出。试编写相应程序。

```

#include <stdio.h>

void digitPrint(int n){
    if(n == 0) //递归出口（基例）
        return ;
    //递归链条
    digitPrint(n / 10);
    //回溯过程中输出
}

```

```
        printf("%d ", n % 10);  
    }  
  
    int main(int argc, char *argv[]) {  
        digitPrint(20020604);  
    }
```

测试结果：

2 0 0 2 0 6 0 4

四、 实验小结：

1. 学会了如何自己定义函数，以及如何形参的定义，在主函数中如何调用，对于模块化的功能，学会了利用函数进行封装。
2. 深刻的理解了函数的实参和形参的值传递方法，以及不同类型函数的返回值对应的情况，并学会了如何巧妙的在函数体中利用好 return 语句；
3. 对于递归算法，理解了递归思想，即在函数中自己调用自己的过程，以及在内存中的原理，也即递归栈，同时，学会了如何设计递归算法以及如何利用递归过程中的回溯解决实际问题；

五、 实验成绩：